Generative AI from First Principles · Lecture 5

Optimizers for Deep Learning

Each optimizer fixes a failure of the one before it

one step size for everything: gradient descent, stochastic gradients, momentum

a step size for each parameter: AdaGrad, RMSProp, Adam; why Adam trains language models

a step for each matrix: Muon

no gradient at all: zeroth-order methods

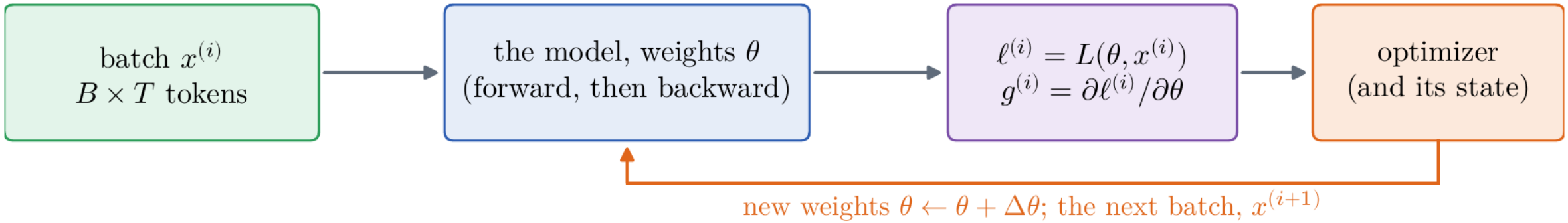
Optimizers for Deep Learning

An optimizer sees a loss and its gradient, batch by batch

what is left: the architecture and the input are fixed; for the optimizer they are abstracted away

one step: a batch of B × T tokens; forward, its loss; backward, its gradient, by backpropagation (the picture)

the whole story: the tutorial, *Optimizers for Deep Learning*; try them in the optimizer explorer



Optimizers tutorial, Figure 1: the optimizer's view of training

Optimizers for Deep Learning

Gradient descent and the condition number

the question: how large may a step be? → **the answer**: below 2/λmax, so the flat directions crawl

$$\mathcal{L}(\theta) = \tfrac{1}{2}(\lambda_1\theta_1^2 + \lambda_2\theta_2^2), \quad \lambda_1 = 1,\ \lambda_2 = 25$$

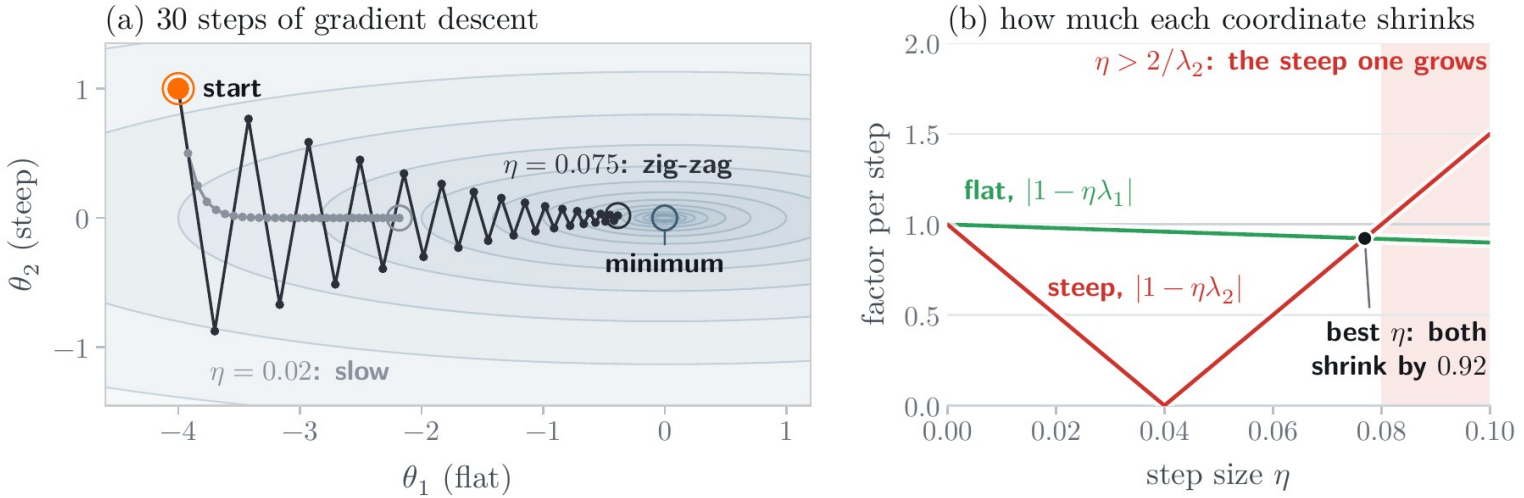
$$\theta_i \leftarrow (1 - \eta\lambda_i)\,\theta_i$$

a quadratic bowl: each coordinate shrinks by its own factor per step

$$\eta < \frac{2}{\lambda_{\max}}, \qquad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$$

the steep direction: sets the limit; past it, the step overshoots and grows

the flat direction: crawls; κ sets the pace, 87 steps for a millionfold at κ = 25



Optimizers tutorial, Figure 2: gradient descent on ½(θ₁² + 25θ₂²), and each coordinate's factor

Optimizers for Deep Learning

Stochastic gradients: the noise floor

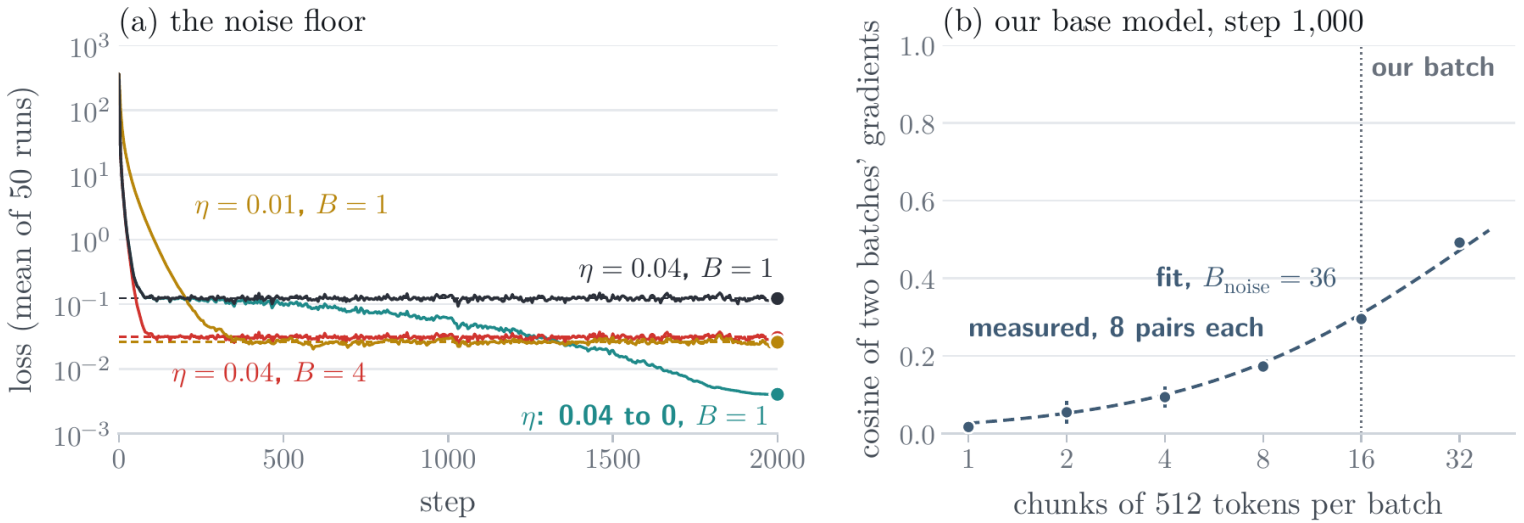
the problem: each step's gradient comes from one batch, and is noisy → **the fix here**: a step that decays, and a large enough batch

a noisy gradient: right on average; its variance falls as 1/B

$$\mathcal{L}_{\text{floor}} \approx \frac{\eta \ \text{tr}\,\Sigma}{4B}$$

a fixed step stops at a floor: a smaller step or a larger batch lowers it equally

decay the step: a decaying η reaches the minimum; on our base model, the noise scale is about 18,000 tokens



Optimizers tutorial, Figure 3: (a) the noise floor; (b) two batches' gradients on our base model

Optimizers for Deep Learning

Momentum and Nesterov's method

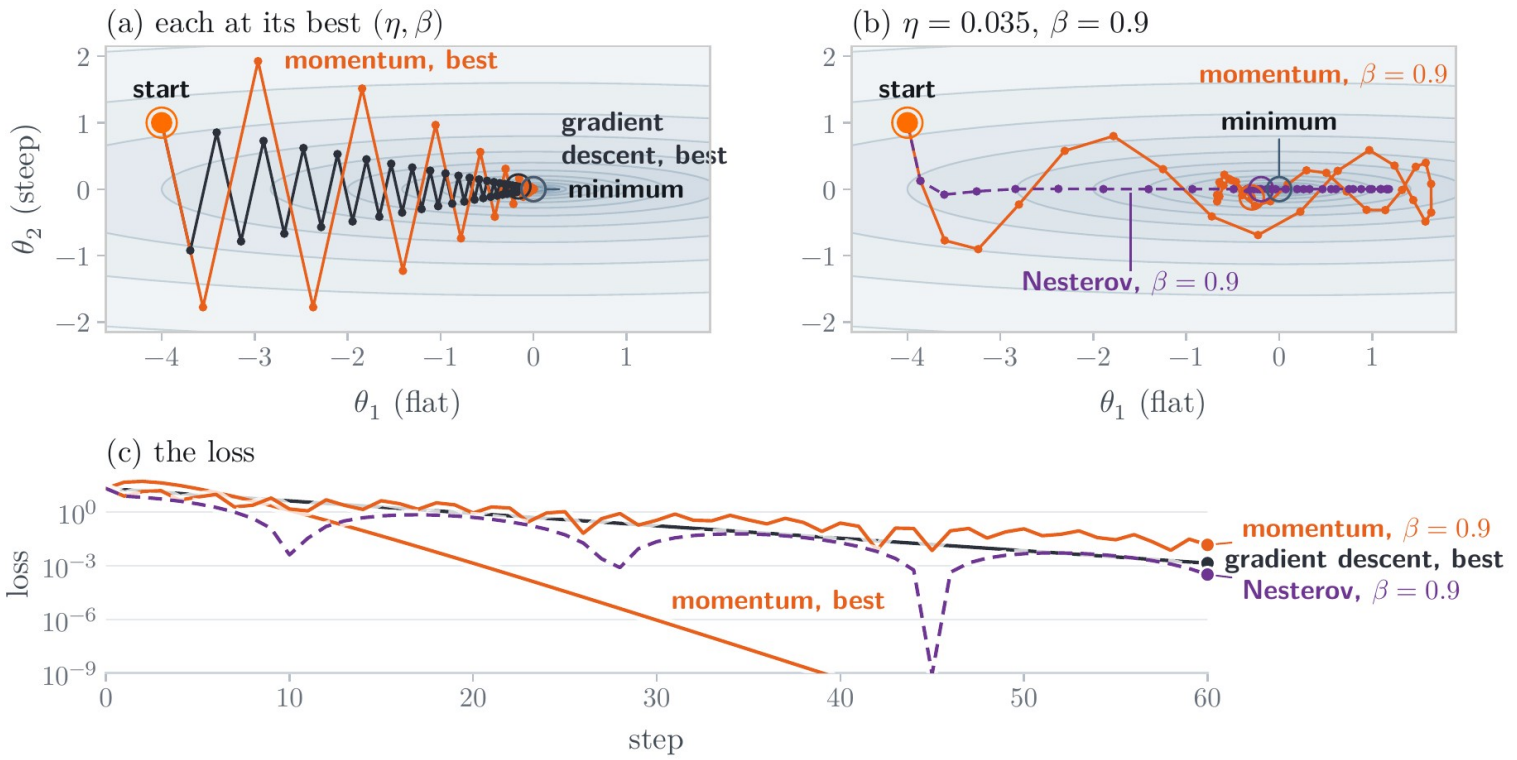
the problem: gradient descent zig-zags across steep directions and crawls along flat ones → **the fix here**: remember recent gradients

$$v \leftarrow \beta v + g, \qquad \theta \leftarrow \theta - \eta v$$

momentum: a running sum of gradients; the zig-zags cancel, steady pulls add up

κ becomes √κ: a millionfold in 26 steps instead of 87

Nesterov's look-ahead: with β = 0.9 the ball overshoots; the gradient ahead brakes it



Optimizers tutorial, Figure 4: gradient descent, momentum, and Nesterov on the bowl of Figure 2

Optimizers for Deep Learning

A step size for each parameter: AdaGrad and RMSProp

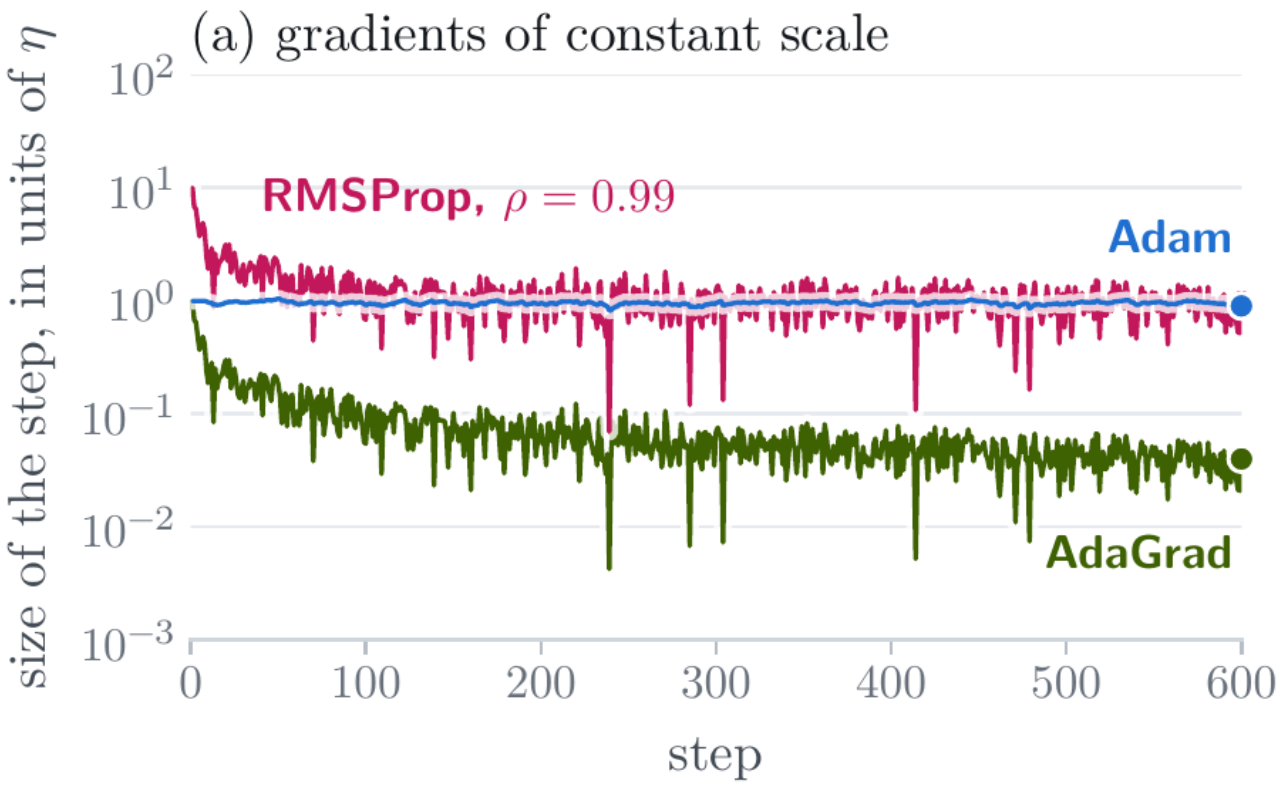
the problem: one step size for gradients that differ in size by orders of magnitude → **the fix here**: divide each gradient by its own typical size

AdaGrad (2011): divide by the root of all past squares; the steps shrink forever

$$s \leftarrow \rho\, s + (1-\rho)\, g^2, \qquad \theta \leftarrow \theta - \eta \frac{g}{\sqrt{s}}$$

RMSProp (2012): a moving average of the squares; the step stays near η

what still fails: the first steps are too large, and each follows one batch's noise



Optimizers tutorial, Figure 6a: step sizes, in units of η

Optimizers for Deep Learning

Adam

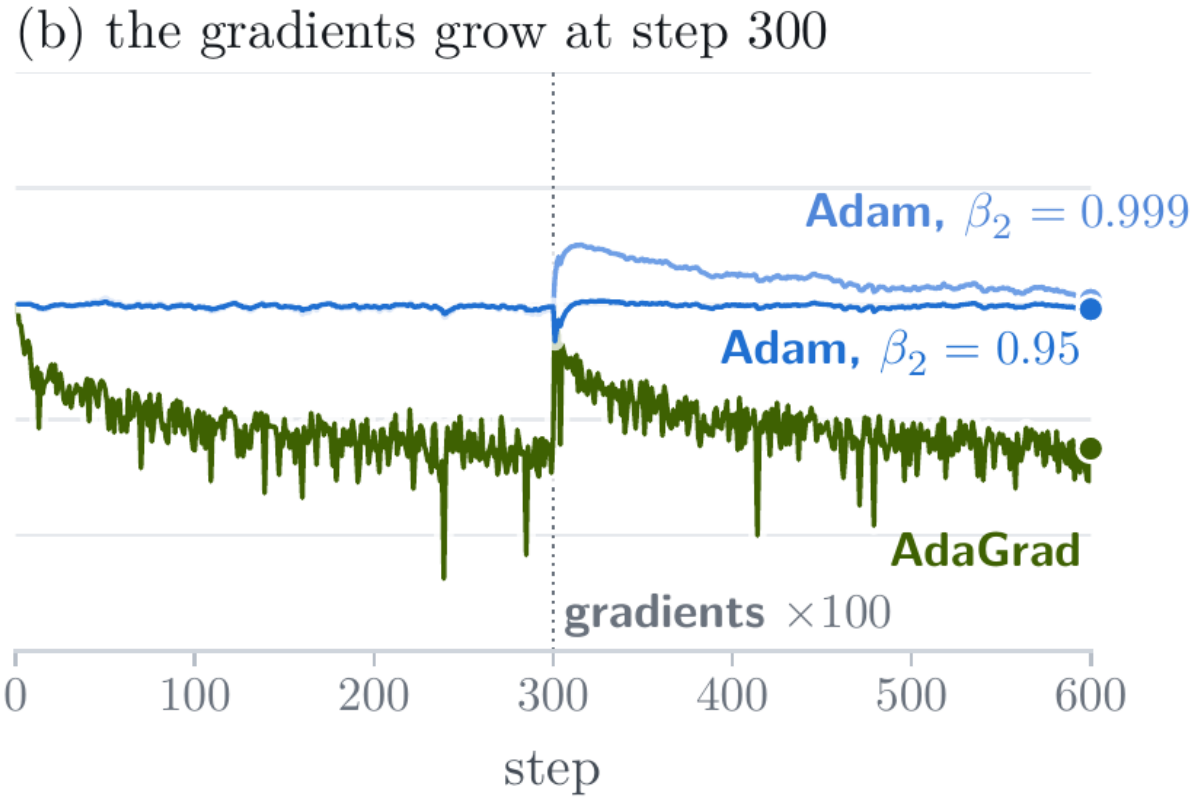
the problem: RMSProp's first steps are too large, and each follows one batch's noise → **the fix here**: average the gradients too, and correct both averages

$$m \leftarrow \beta_1 m + (1-\beta_1)\, g, \qquad v \leftarrow \beta_2 v + (1-\beta_2)\, g^2$$
$$\hat{m} = \frac{m}{1-\beta_1^t}, \qquad \hat{v} = \frac{v}{1-\beta_2^t}$$
$$\theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

Adam (2015): momentum's average over RMSProp's scale; a step of about η in every coordinate

its settings: β_1 = 0.9 and β_2 = 0.95 average about 10 and 20 steps; ε avoids dividing by zero

β_2 = 0.999 remembers too long: when the gradients grow, it overshoots for hundreds of steps



Optimizers tutorial, Figure 6b: the gradients grow a hundredfold at step 300

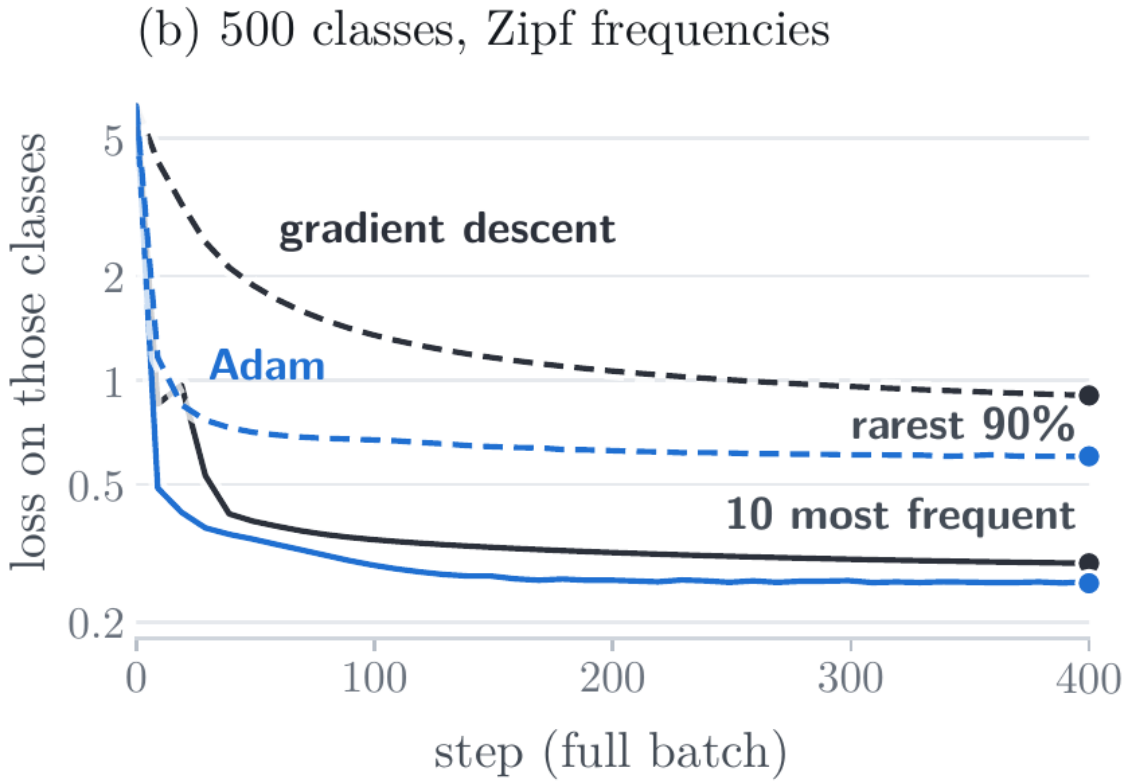
Optimizers for Deep Learning

Why Adam, not momentum, for language models

the question: momentum can beat Adam on a bowl; why does Adam win on Transformers? → **the answer**: their gradients differ in size by orders of magnitude

our base model: gradient sizes from 7.7×10^{-8} (rare tokens' rows) to 1.5×10^{-3}; one step size cannot fit them all

rare words: gradient descent learns rare classes slowly; Adam does not



Optimizers tutorial, Figure 5b: rare classes, gradient descent against Adam

Optimizers for Deep Learning

A step for each matrix

the problem: a matrix's gradient pushes hard along a few directions and barely along the rest → **the fix here**: a step of the same size along every direction

$$G = USV^\top = \sum_i s_i \, \mathbf{u}_i \mathbf{v}_i^\top$$

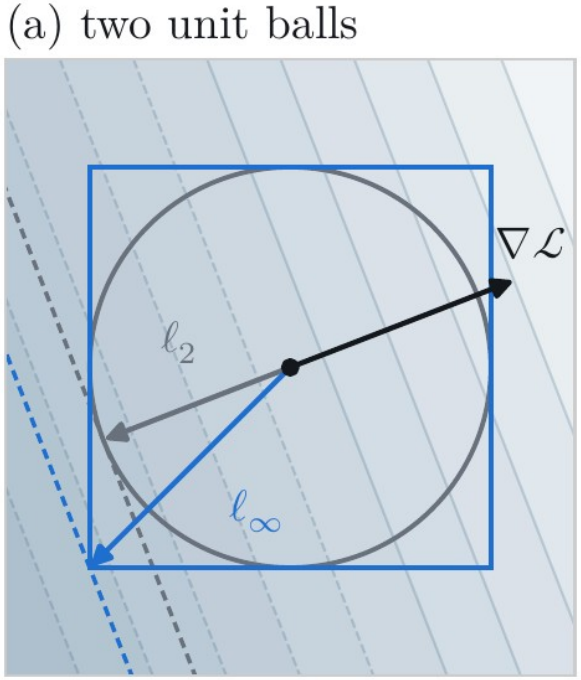
a gradient's directions: our query gradient's singular values span a factor of about 40,000

$$G = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad \operatorname{sign} G = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \qquad UV^\top = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Adam on a matrix: every entry moves by η, and the weaker direction is lost

$$M = USV^\top \longrightarrow O = UV^\top$$

steepest in the spectral norm: $UV^\top$, every singular value set to 1



Optimizers tutorial, Figure 7a: two unit balls and their steepest steps

Optimizers for Deep Learning

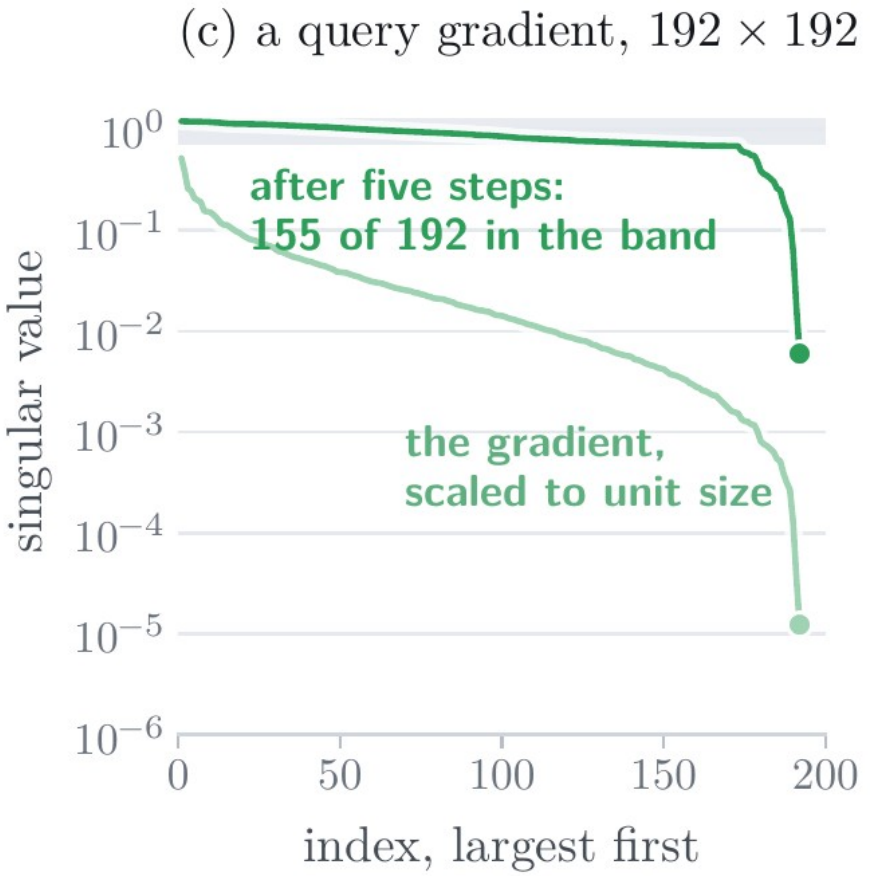
Muon

Muon (2024): the momentum's UV^T without an SVD, by five Newton–Schulz steps

a quintic, not the cubic: it lifts small singular values about 480-fold, into 0.7 to 1.3

our query gradient: 155 of 192 singular values in that band after five steps

the rest keeps AdamW: the token table, the output layer, and the vectors



Optimizers tutorial, Figure 7c: a query gradient's singular values

Optimizers for Deep Learning

Our model, five optimizers

our own experiment: Lecture 4's shape (4 blocks, d = 128), our 4,096 tokens; 800 steps of 16 × 256 tokens; each optimizer at several peak rates

each fix lowers the loss: Muon 0.3 nats below Adam

best rates a hundredfold apart: 1 for SGD, 0.01 for Adam; Adam's step is about η

Muon: it separates from Adam within about 100 steps, and is the least sensitive to its rate

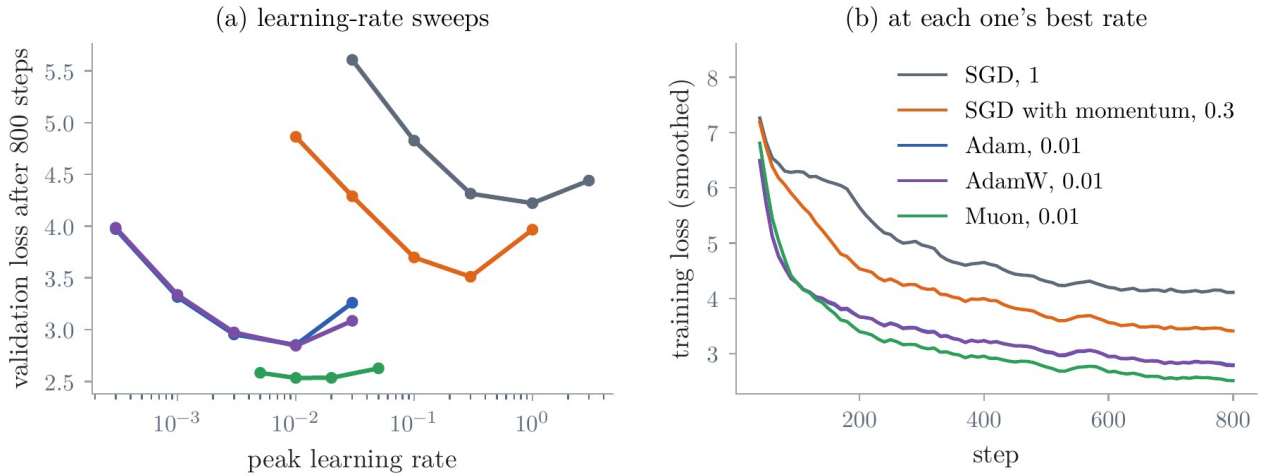


Figure 22: (a) validation loss after 800 steps; (b) training loss at each one's best rate

Optimizers for Deep Learning

No gradient at all: zeroth-order methods

the problem: a gradient and Adam's state cost about 16 bytes per parameter, running the model 2 → **the fix here**: estimate the gradient from forward passes alone

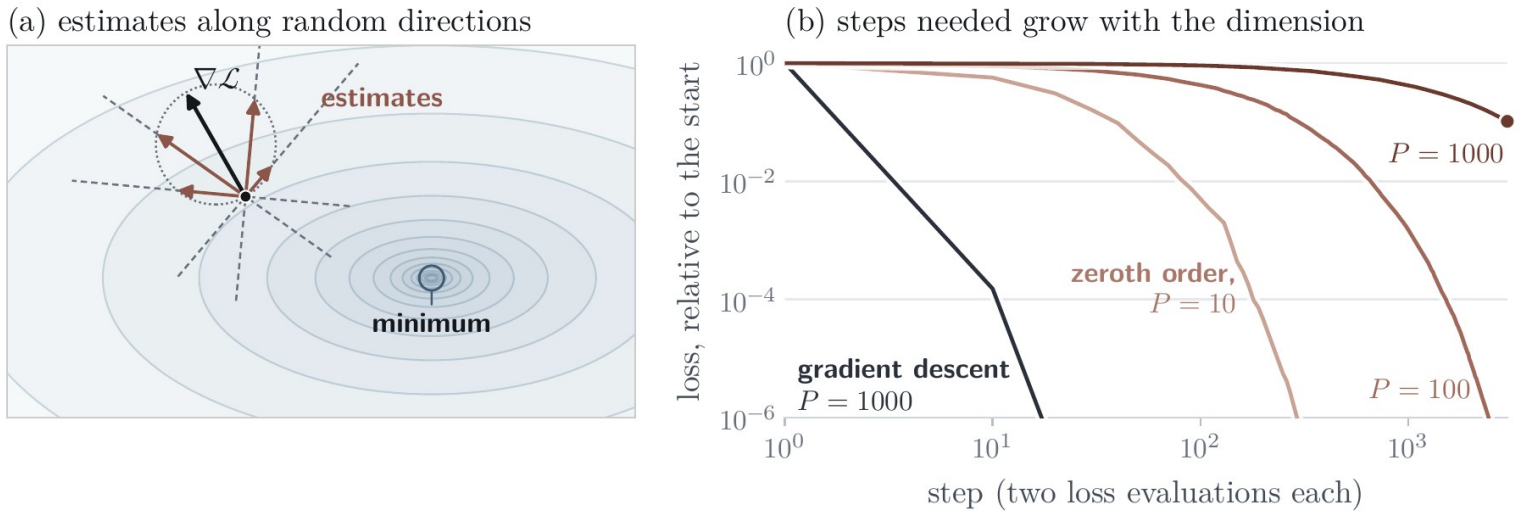
$$\hat{\mathbf{g}} = \frac{\mathcal{L}(\theta + \varepsilon\mathbf{z}) - \mathcal{L}(\theta - \varepsilon\mathbf{z})}{2\varepsilon}\mathbf{z} \;\approx\; (\mathbf{z}^\top\mathbf{g})\,\mathbf{z}$$

the slope along a random direction: two forward passes; right on average

about P times the steps: all but 1/P of each estimate is noise

MeZO (2023): fine-tuning in the memory of inference; 30 billion parameters on one GPU

evolution strategies: perturbed models scored by any reward; ahead of PPO and GRPO on Countdown



Optimizers tutorial, Figure 8: (a) the estimate's directions; (b) steps against the number of parameters

Optimizers for Deep Learning

Which optimizer when

optimizer	state per parameter	fixes	still fails when
gradient descent	none		curvatures differ (κ steps)
SGD, a decaying η	none	the noise floor	as above
momentum, Nesterov	1	κ becomes $\sqrt{\kappa}$	gradient scales differ
AdaGrad	1	scales differ	steps shrink forever
RMSProp	1	AdaGrad's shrinking	noisy, too large at first
Adam, AdamW	2	RMSProp's noise and first steps	a matrix's directions
Muon	1, matrices only	a matrix's directions	applied to tables and vectors
zeroth order	none	memory; any reward	about P times the steps

Optimizers tutorial, §8

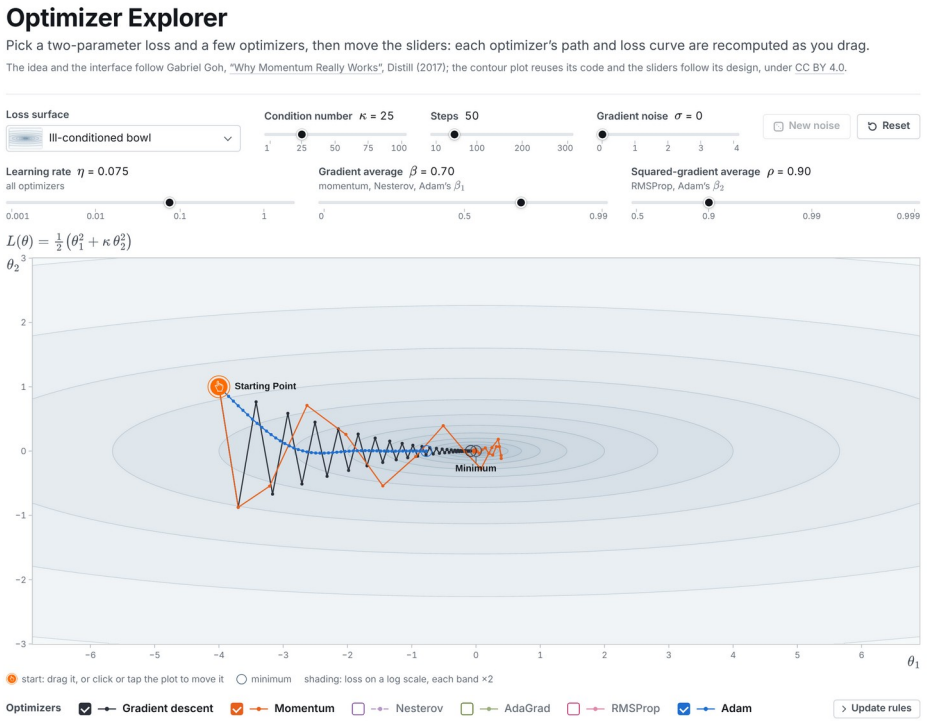
in 2026: AdamW, β = (0.9, 0.95), with a warmup and a decay; or Muon for the matrices inside the blocks

Optimizers for Deep Learning

Try them: the optimizer explorer

six optimizers: gradient descent to Adam, side by side; one η, one β, and one ρ for all

what to try: η just above 2/κ; the noise floor; gradients of very different sizes; the rotated bowl



Slide figure: the optimizer explorer, materials/week-05/optimizer-explorer.html